

UNIVERSIDAD LA SALLE



Facultad de Ingenierías y Arquitectura
Escuela Profesional de Ingeniería de Software

Curso: Calidad de Software
Docente: Maribel Molina Barriga

PRÁCTICA DE LABORATORIO 02

Matriz de hallazgos y ADR

Proyecto OLA — Observatorio Litoral de
Anomalías térmicas

*Matriz de hallazgos priorizada del módulo «Alertas y avisos» y
registro de decisión de arquitectura ADR-001*

Integrantes

Frederick Mares Graos
Jhordan Huamani Huamani
Jorge Ortiz Castañeda
Miguel Angel Flores Leon
Piero Adrian Delgado Chipana

Arequipa, octubre de 2026

Índice

1. Matriz de hallazgos	2
2. ADR-001: Separar el envío de avisos en canales, almacén y despachador	4
2.1. Contexto	4
2.2. Opciones consideradas	4
2.3. Decisión	5
2.4. Consecuencias	5
2.5. Verificación	7
2.6. Referencias	7

1. Matriz de hallazgos

La revisión del módulo «Alertas y avisos» (commit d5c054f) produjo 12 hallazgos, que la matriz de la página siguiente ordena por prioridad (**E-04**). De ellos, 5 quedan resueltos por el parche de refactorización (H-01 a H-05), 5 siguen pendientes (H-06, H-07, H-09, H-10 y H-12) y 2 se aceptaron con su razón (H-08 y H-11). La misma matriz, con sus fórmulas, está en la hoja «Hallazgos» de `Matriz_Hallazgos_OLA.xlsx`.

Escala. Severidad: Alta = impacto 4–5; Media = 3; Baja = 1–2. Impacto: 1 = cosmético a 5 = bloquea cambios previsibles o impide probar. Esfuerzo: 1 = menos de una hora a 5 = varios días. Prioridad: orden por impacto $\times$ 10 – esfuerzo.

Nota sobre la numeración. La matriz corresponde a la Tabla 3 del informe técnico; conserva su título original.

Tabla 3: E-04. Matriz de hallazgos, ordenada por prioridad

ID	Hallazgo	Principio / atributo	Sev.	Imp.	Esf.	Prior.	Evidencia	Decisión	Estado
H-01	notification_service reúne cinco responsabilidades (planificar avisos, persistirlos, redactar el mensaje, enviarlo y marcar la lectura) y construye 4 sentencias SQL sin pasar por el repositorio.	SRP · respeto de capas · modificabilidad	Alta	5	3	1	E-01, E-03, E-05. notification_service.py: 46–55, 85–90, 104–110, 125–130. LOC 141, CCsum 20.	Refactorizar: el servicio solo coordina; la persistencia pasa al repositorio y el envío a un despachador (R2).	Resuelto en el parche
H-02	Los canales no son extensibles: CANALES es una tupla fija, send_pending solo sabe enviar correo y el repositorio filtra por EMAIL en dos consultas. Un canal nuevo obliga a modificar el servicio, el repositorio, el router y las dependencias.	OCP · modificabilidad	Alta	5	3	1	E-01, E-05, E-10. notification_service.py: 36, 153–179; notifications_repo.py: 55–71, 74–80. En el escenario webhook cambian 6 módulos existentes.	Rediseñar con un contrato Channel y un registro de canales (R1). count_by_status, que solo alimenta el resumen del envío, sigue contando el correo.	Resuelto en el parche
H-05	La lógica de envío (reintentos, registro del fallo, límite de la tanda) no puede probarse sin PostgreSQL: tiene 0 pruebas unitarias y 34 de integración.	Comprobabilidad	Alta	4	3	3	E-09. tests/integration/test_notifications.py: 34 pruebas, 12.5–12.9 s en 5 ejecuciones.	Extraer la lógica a NotificationDispatcher, que depende de los protocolos NotificationStore, Channel y Clock (R2).	Resuelto en el parche
H-03	El bloque insert ... on_conflict_do_nothing ... returning y el diccionario de la fila están duplicados en create_for_events y notify_open_alerts_of.	DRY · mantenibilidad	Media	3	1	4	E-05. notification_service.py: 71–90 y 114–131.	Unificar en notifications_repo. insert_pending y domain.notices.plan_rows (R2).	Resuelto en el parche
H-04	La abstracción Clock (con SystemClock y FixedClock) no la importa ningún módulo de producción (Ca = 0). notification_service llama a datetime.now(UTC) en 3 lugares.	DIP · comprobabilidad	Media	3	1	4	E-01, E-03. clock.py: 15–31; notification_service.py: 175, 187, 194.	Injectar Clock en el despachador y en la marca de lectura (R2).	Resuelto en el parche
H-10	El router coordina dos servicios (evaluar y registrar avisos), accede a 4 repositorios directamente y repite 4 veces la búsqueda de laboratorio con su error 404.	SRP · DRY · respeto de capas	Media	3	2	6	E-03. routers/alerts.py: 26–34, 47–58, 78–83; subscriptions.py: 39–44, 62–67. Ce = 10, I = 0.91.	Crear una dependencia LaboratoryDep y un caso de uso evaluate_and_notify.	Pendiente
H-06	alert_service construye 4 sentencias SQL en lugar de usar alerts_repo: la consulta y el borrado de los episodios obsoletos, el upsert de los vigentes y la consulta de los identificadores existentes.	Respeto de capas · mantenibilidad	Media	3	3	7	E-01, E-03. alert_service.py: 60–105 y 163.	Mover el upsert y las consultas a alerts_repo.	Pendiente
H-07	Smtplib recibe el objeto Settings completo (21 campos) y usa 6: acoplamiento de sello. Por esa importación, todo módulo que use mail carga pydantic y la configuración.	ISP · acoplamiento	Baja	2	1	8	E-03, E-07. mail.py: 14, 31–46; config.py: 23–60.	Pasar una configuración SMTP propia de 6 campos.	Pendiente
H-09	El estado se recibe como str y se traduce con dos diccionarios paralelos; AlertState duplica a ThermalState y exige un conversor.	Obsesión por primitivos · DRY	Baja	2	2	9	messages.py: 16–17, 51–52; alert_service.py: 40–41.	Usar ThermalState en los mensajes y un solo enumerado.	Pendiente
H-12	Los límites de transacción son inconsistentes: create_for_events, send_pending, mark_read y mark_all_read confirman dentro del servicio, pero notify_open_alerts_of no lo hace y confirma el router que lo llama.	Consistencia · mantenibilidad	Baja	2	2	9	notification_service.py: 92, 96–131, 178, 188, 197; routers/subscriptions.py: 50–51.	Fijar la transacción en un solo nivel (router o caso de uso). El parche lo documenta en notify_open_alerts_of, pero no lo corrige.	Pendiente
H-11	El canal se guarda como tipo enumerado de PostgreSQL: añadir un canal exige una migración en cualquier diseño.	Modificabilidad	Baja	2	3	11	E-10. models.py: 309–316; migración 9c022526c786: 44–46.	Aceptar; reevaluar si se añaden canales con frecuencia.	Aceptado
H-08	RecordingMailer, un doble de prueba, vive en un módulo de producción.	Cohesión	Baja	1	1	12	mail.py: 49–59.	Mover a tests/ cuando se toquen las pruebas.	Aceptado por ahora

Severidad: Alta = impacto 4–5; Media = 3; Baja = 1–2. Impacto: 1 = cosmético ... 5 = bloquea cambios previsibles o impide probar. Esfuerzo: 1 = menos de una hora ... 5 = varios días. Prioridad: orden por Impacto × 10 – Esfuerzo (empates en la misma posición). «Resuelto en el parche»: corregido en refactorizacion_avisos.patch y validado (472 pruebas pasan); falta aplicarlo en una rama (ADR-001).

2. ADR-001: Separar el envío de avisos en canales, almacén y despachador

Estado	Aceptado (pendiente de aplicar el parche en una rama)
Fecha de la propuesta	2026-10-05
Fecha de aceptación	2026-10-06
Proyecto	OLA — Observatorio Litoral de Anomalías térmicas
Alcance	Backend, módulo «Alertas y avisos» (RF-01, RF-03)
Versión evaluada	commit d5c054f (2026-09-13)
Decisores	Equipo OLA; aprobó Miguel Angel Flores Leon (Product Owner)
Hallazgos que resuelve	H-01, H-02, H-03, H-04, H-05

2.1 Contexto

`services/notification_service.py` reúne cinco responsabilidades: decide qué avisos corresponden a cada episodio, los inserta en la base, redacta el mensaje, lo envía por correo y marca los avisos como leídos. La revisión de diseño encontró cuatro problemas que nacen de ahí:

1. **Los canales no son extensibles.** CANALES es una tupla fija, `send_pending` solo sabe enviar correo y `notifications_repo` filtra por EMAIL en dos consultas. Añadir un canal obliga a modificar el servicio, el repositorio, el router y las dependencias (H-02).
2. **El servicio construye SQL.** Contiene 4 sentencias `select / insert ... on_conflict` y repite dos veces el mismo bloque de inserción (H-01, H-03).
3. **La lógica de envío no se puede probar sin PostgreSQL.** Los reintentos, el registro del fallo y el límite de la tanda tienen 0 pruebas unitarias y 34 de integración, que tardan entre 12.5 y 12.9 s (H-05).
4. **La abstracción Clock existe pero no se usa.** Ningún módulo de producción la importa; el servicio llama a `datetime.now(UTC)` en 3 lugares (H-04).

La SRS retiró el SMS de RF-03 por su costo por mensaje, de modo que hoy solo existen el correo y el aviso dentro de la aplicación. El equipo considera que el cambio más previsible del módulo es añadir otro canal de aviso.

Restricciones: el equipo no puede ejecutar una reescritura grande antes del Hito 2, y las 442 pruebas existentes del backend deben seguir pasando sin modificarlas.

2.2 Opciones consideradas

Opción	Descripción
A. Puertos y canales	Un contrato Channel por canal y un registro de canales; un contrato NotificationStore para la persistencia; un NotificationDispatcher que recibe almacén, canales y reloj.
B. Cambio mínimo	Mover el SQL al repositorio y añadir un condicional por canal dentro de <code>send_pending</code> .
C. Cola asíncrona	Publicar cada aviso en una cola y enviarlo con un trabajador en segundo plano.

Matriz ponderada (1 = peor, 5 = mejor; en esfuerzo y riesgo, 5 es menor esfuerzo y menor riesgo):

Criterio	Peso	A	B	C
Modificabilidad	30 %	5	3	5
Comprobabilidad	25 %	5	2	4
Esfuerzo de implementación	20 %	3	5	1
Riesgo de regresión	15 %	4	5	2
Comprensibilidad para el equipo	10 %	3	4	2
Total ponderado		4.25	3.55	3.20

2.3 Decisión

Se adopta la **opción A**.

- `domain/notices.py` define los conceptos del aviso sin dependencias de infraestructura: los enumerados de canal, tipo y estado (que antes estaban en `db/models.py`), `plan_rows` y `compose`.
- `services/channels.py` define el contrato `Channel`, los canales `EmailChannel` e `InAppChannel`, el `ChannelRegistry` y la función `default_channels`, único punto donde se decide qué canales tiene el sistema.
- `services/notification_dispatcher.py` contiene `NotificationDispatcher`, que depende solo de los protocolos `NotificationStore`, `Channel` y `Clock`.
- `repositories/notifications_repo.py` concentra la inserción (`insert_pending`) y aporta `SqlNotificationStore`, la implementación del almacén sobre PostgreSQL.
- `services/notification_service.py` queda como coordinador: fija el límite de cada transacción y conserva sus firmas públicas (`send_pending` sigue existiendo y delega en `dispatch_pending`), de modo que las pruebas existentes no cambian.
- `api/deps.py` añade `get_channels`, y `api/routers/alerts.py` recibe los canales por inyección (`ChannelsDep`) en lugar del `Mailer`. Es el único router que cambia, en 3 líneas.

2.4 Consecuencias

Positivas

- **Modificabilidad.** Se añadió un canal de prueba (una pasarela por *webhook*) con la misma especificación en ambos diseños. Los módulos existentes modificados bajan de 6 a 4 y las líneas de código existente reescritas, de 13 a 4. El servicio, el despachador, el repositorio y el router no cambian. La complejidad del bucle de envío se mantiene en 4, mientras que en el diseño actual sube a 6.
- **Comprobabilidad.** La lógica de envío pasa de 0 a 12 pruebas unitarias; con las de canales y avisos son 30, que se ejecutan en 0.04 s sin base de datos. `domain/notices.py` se importa sin ninguna biblioteca externa, y el despachador y los canales, sin SQLAlchemy, FastAPI ni `psycopg`.

- **Cohesión.** `notification_service` baja de 141 a 73 líneas y su complejidad acumulada, de 20 a 12; deja de construir SQL (4 sentencias a 0).
- Clock pasa a usarse (Ca: 0 a 2).

Negativas

- **Más código.** El módulo revisado crece de 1 011 a 1 173 líneas (+16 %) y aparecen 3 módulos nuevos.
- **Más indirección.** Seguir un envío exige leer cuatro archivos en lugar de uno. `notification_service` depende ahora de 8 módulos internos en lugar de 5.
- **No reduce las pruebas que hay que actualizar al añadir un canal.** En el escenario del *webhook* dejan de pasar las mismas 11 pruebas en ambos diseños, porque cuentan los avisos por episodio o el resumen del envío, que cambian por especificación. El diseño propuesto suma una más, `test_solo_el_correo_requiere_envio`, que fija qué canales requieren envío.
- **Los canales siguen cargando la configuración completa.** `channels` importa `mail`, y `mail` importa `Settings`; por eso el despachador y los canales cargan `pydantic` y `smtplib` (H-07).
- **El enumerado de canal sigue en PostgreSQL.** Añadir un canal exige una migración en cualquier diseño (H-11).
- Quien incorpore un canal debe registrarlo en `default_channels`; si lo olvida, sus avisos quedan pendientes. La prueba `test_los_canales_por_defecto_cubren_todos_los_que_se_guardan` falla en ese caso.

Pendiente

- Aplicar `refactorizacion_avisos.patch` en una rama del repositorio y fusionarla.
- Aplicar el mismo criterio a `alert_service`, que también construye SQL (H-06).
- Fijar el límite de las transacciones en un solo nivel (H-12): `notify_open_alerts_of` sigue sin confirmar y lo hace el router.
- `count_by_status` sigue contando solo el correo.

2.5 Verificación

Comprobación	Resultado
Pruebas unitarias previas	212 de 212 pasan
Pruebas de integración previas	230 de 230 pasan, con el dataset montado en /dataset
Pruebas unitarias nuevas	30 de 30 pasan en 0.04 s
<code>ruff check .</code> con la configuración del proyecto	Sin observaciones, antes y después
<code>mypy src</code> (estricto, con el complemento de <code>pydantic</code>)	0 errores antes (61 archivos) y después (64 archivos)
Cobertura de sentencias del backend	95.11 % antes y 95.39 % después
Cobertura de los módulos del alcance	95.63 % antes y 96.32 % después
El parche se aplica sobre <code>d5c054f</code>	<code>git apply --check</code> sin conflictos; 472 pruebas pasan

Entorno de la verificación: imagen `ola-api` del proyecto (Python 3.13.15, `pytest` 9.1.1, `ruff` 0.16.6, `mypy` 2.3.1, `bcrypt` 5.0.0) contra PostgreSQL 17.11, con la configuración de `.env.example` y secretos propios. La base de datos corrió en un contenedor desechable en lugar del servicio `db` de `compose.yml`.

2.6 Referencias

- Informe de la Práctica de Laboratorio 02, secciones 3 a 6 (evidencias **E-01** a **E-11**).
- `refactorizacion_avisos.patch`: el cambio completo.
- `escenario_webhook/`: el escenario de cambio aplicado a los dos diseños.
- `evidencias/`: salidas en bruto de `pytest`, `ruff`, `mypy`, la cobertura y las métricas.
- M. Nygard, «Documenting Architecture Decisions», 2011.
- R. C. Martin, *Clean Architecture*, Pearson, 2017, caps. 8 (OCP) y 11 (DIP).
- ISO/IEC 25010:2023, característica de mantenibilidad.